



AI데이터분석

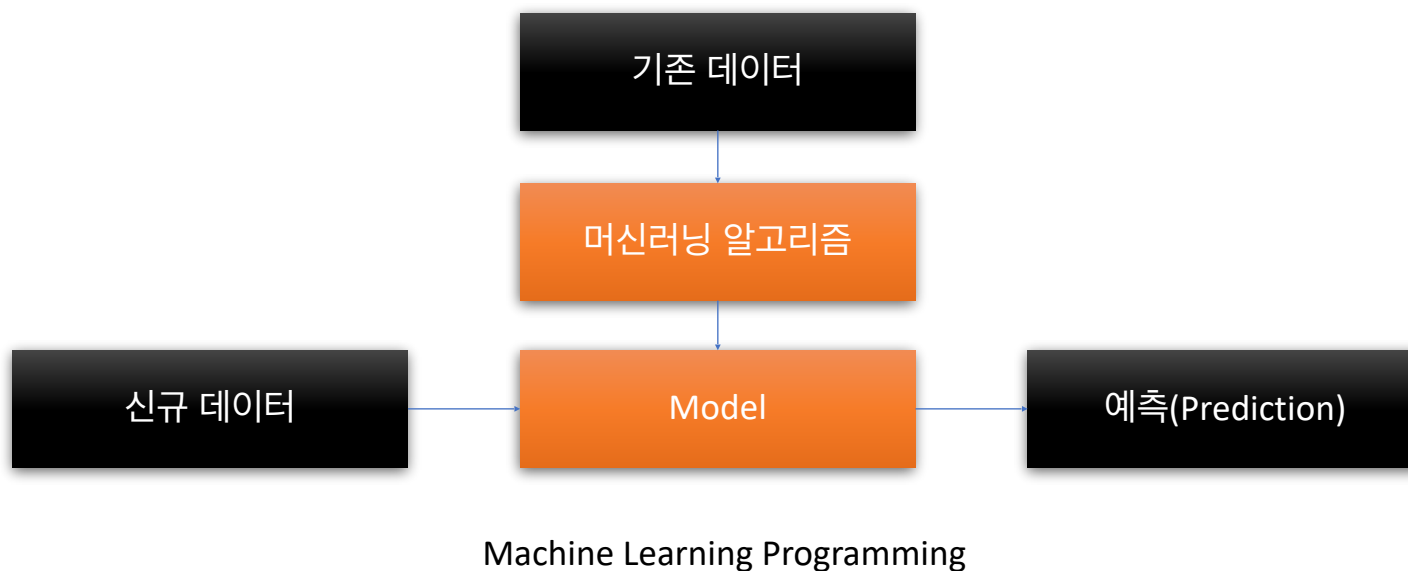
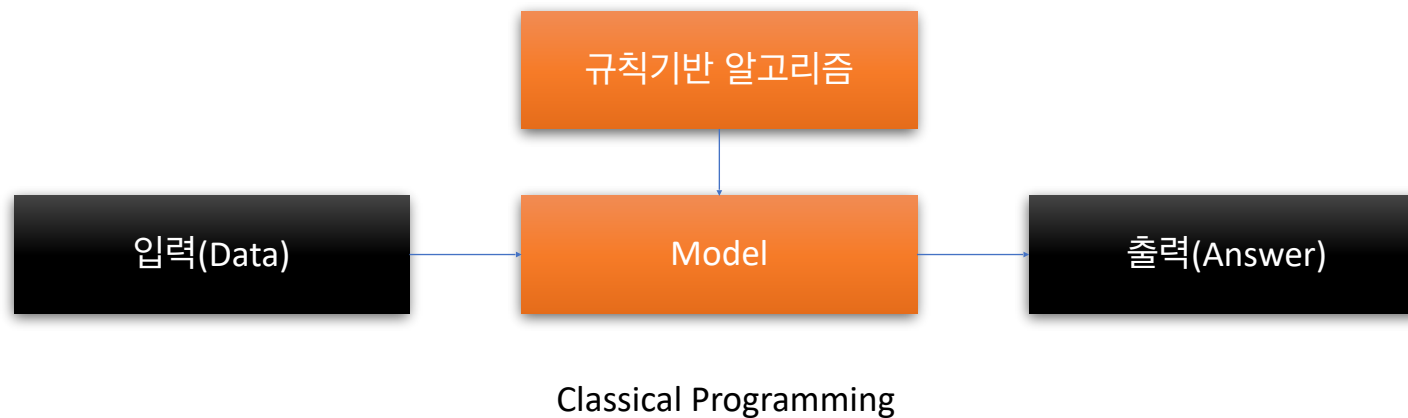
Contents

- **AI데이터분석 기초 프로그래밍**
 - 파이썬 프로그래밍 기초
 - 절차지향 프로그래밍
 - 파이썬 자료구조
 - 객체지향 프로그래밍
- **AI데이터분석 라이브러리**
 - 텍스트 데이터 처리
 - 넘파이 배열
 - 맷플롯립
 - 판다스
- **AI데이터분석 응용**
 - 데이터 수집
 - 데이터 전처리
 - 데이터분석과 머신러닝

3. AI데이터분석 응용

3-3. 데이터분석과 머신러닝

- 인공지능 (Artificial Intelligence)
 - 인간처럼 학습하고 추론하는 프로그램 연구
- 기계학습 (Machine Learning)
 - 인공지능의 한 분야
 - 프로그래밍 없이 스스로 학습하는 프로그램 연구
- 딥러닝 (Deep Learning)
 - 기계 학습의 한 분야
 - 인공 신경망 등을 사용하여 빅데이터로부터 학습하는 프로그램 연구





출처 : <https://asthtls.tistory.com/380>



출처 : <https://busy.org/@urobotics/5bksow>

지도 학습(Supervised Learning)

- 컴퓨터는 "교사"에 의해 주어진 예제와 정답(혹은 레이블)을 제공받음
- 지도학습의 목표
 - 입력을 출력에 매핑하는 일반적인 규칙을 학습



출처 : http://terms.tta.or.kr/dictionary/dictionaryView.do?word_seq=168096-9

- 지도학습

- 과거데이터 $\Rightarrow$ 학습 $\Rightarrow$ 모델(독립변수와 종속변수의 관계식) 생성
- 공식의 대중화

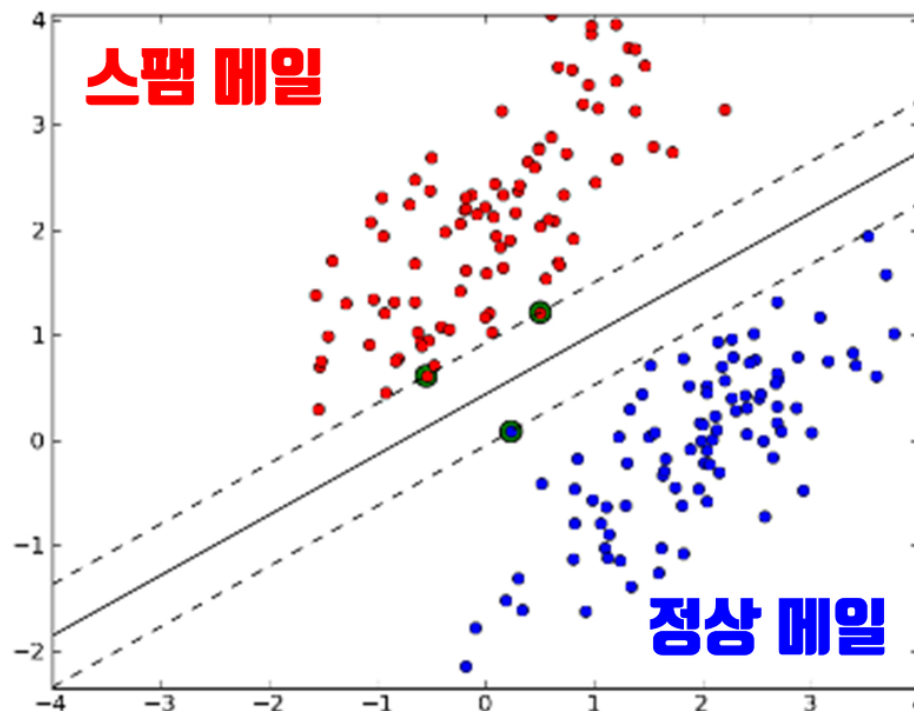
- 선형회귀

- 선형 관계식으로 모델 생성
- 모델에 신규 데이터 입력 $\Rightarrow$ 예측치 출력



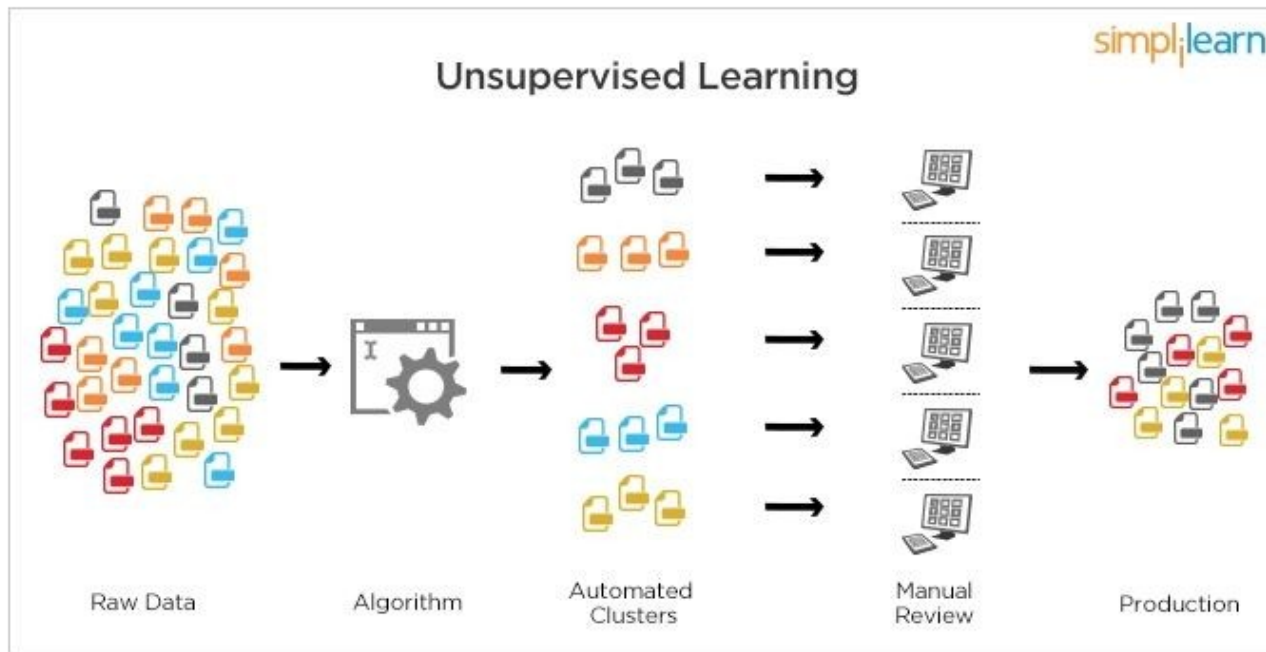
출처 : <https://opentutorials.org/module/4916/28938>

- 앞에 나왔던 식 $y = f(x)$ 에서 출력 y 가 이산적(discrete)인 경우
 - 분류 문제, 인식 문제
- 분류에서는 입력을 2개 이상의 클래스로 나누는 것



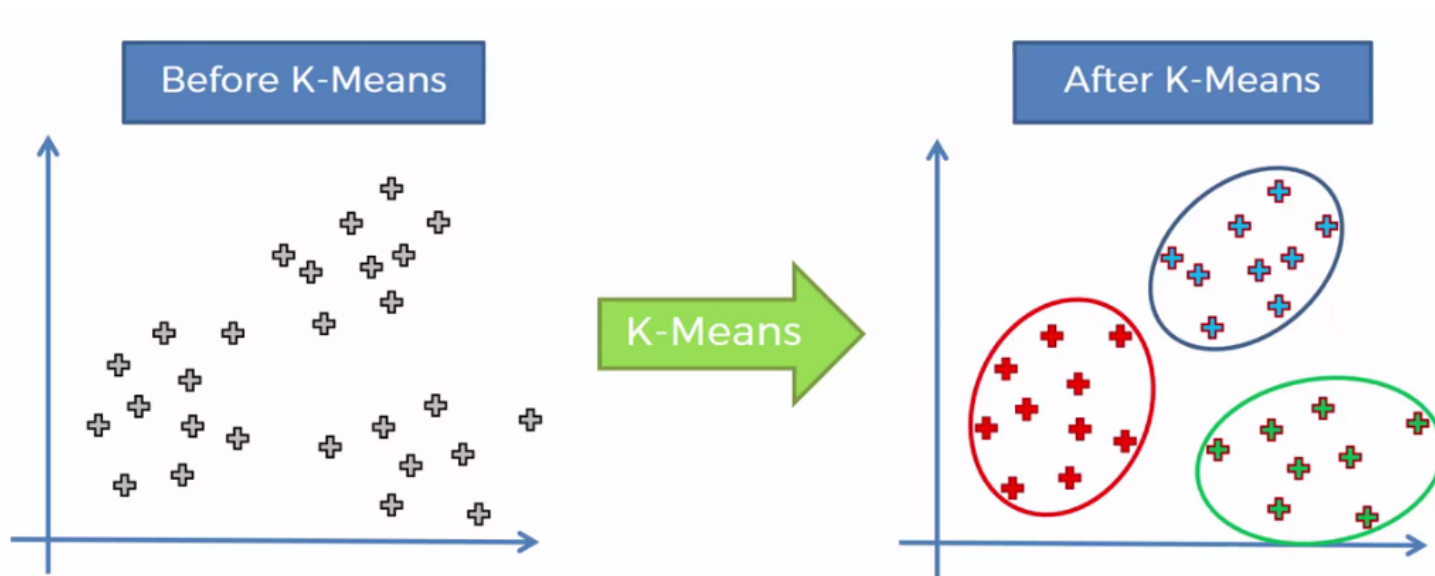
비지도 학습(Unsupervised Learning)

- “교사” 없이 컴퓨터가 스스로 입력들을 분류
 - 식 $y = f(x)$ 에서 레이블 y 가 주어지지 않는 것
- 데이터들의 상관도를 분석
 - 유사한 데이터 모으기



출처 : www.simplilearn.com

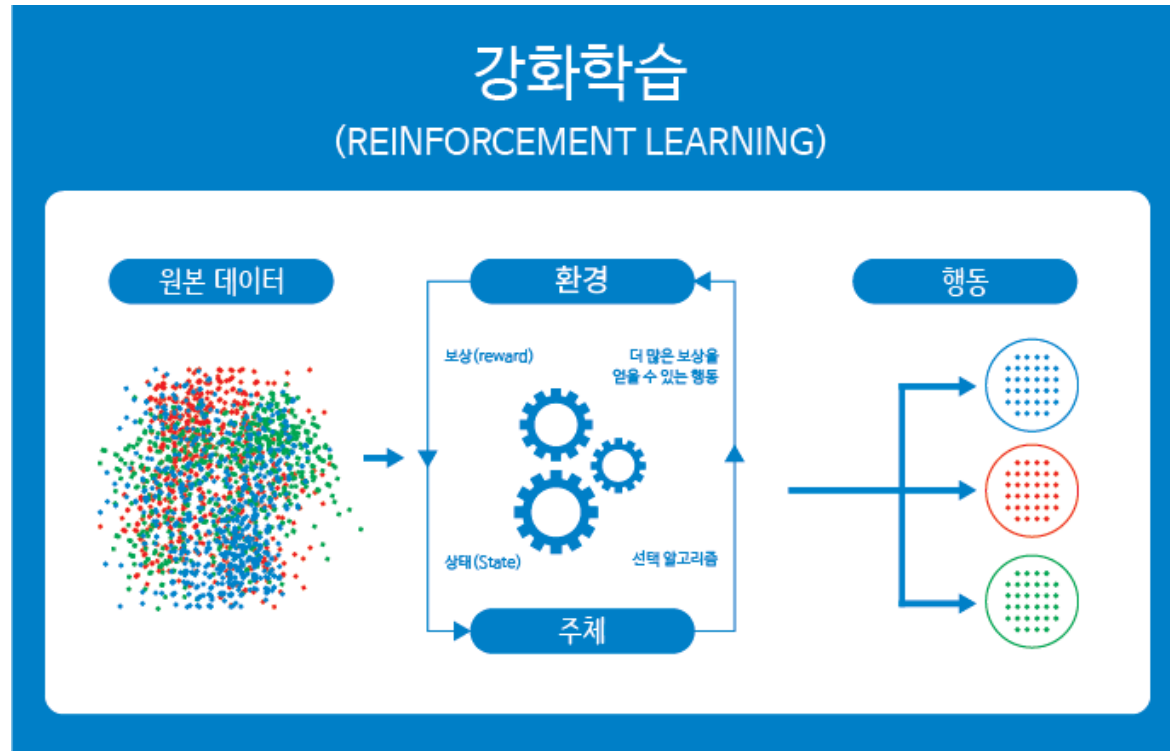
- 가장 대표적인 비지도 학습
 - 클러스터링(군집화, clustering)
 - 데이터간 거리를 계산하여서 입력을 몇 개의 그룹으로 나누는 방법



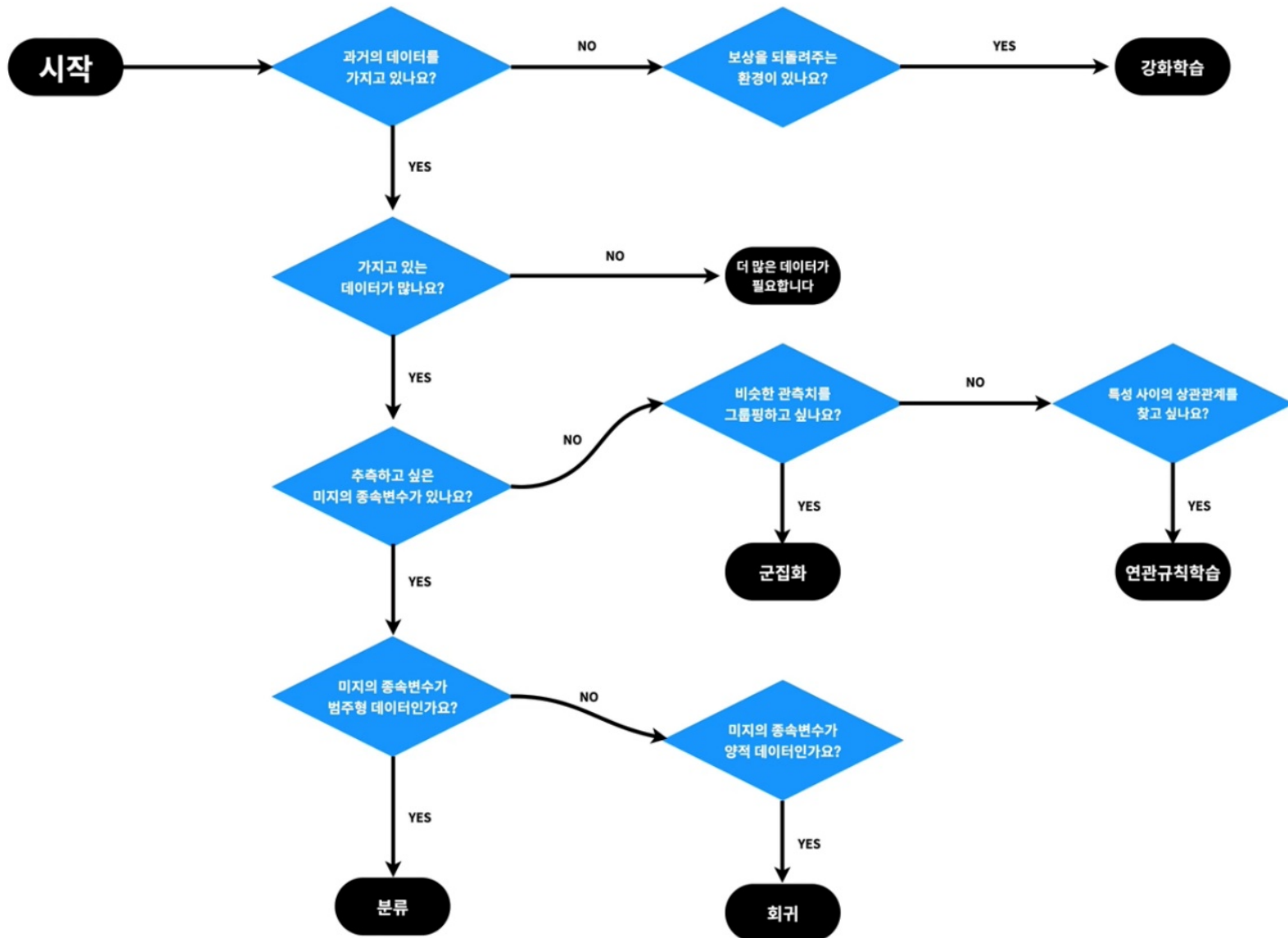
출처 : <https://techblog-history-younghunjo1.tistory.com/83>

강화 학습(Reinforcement Learning)

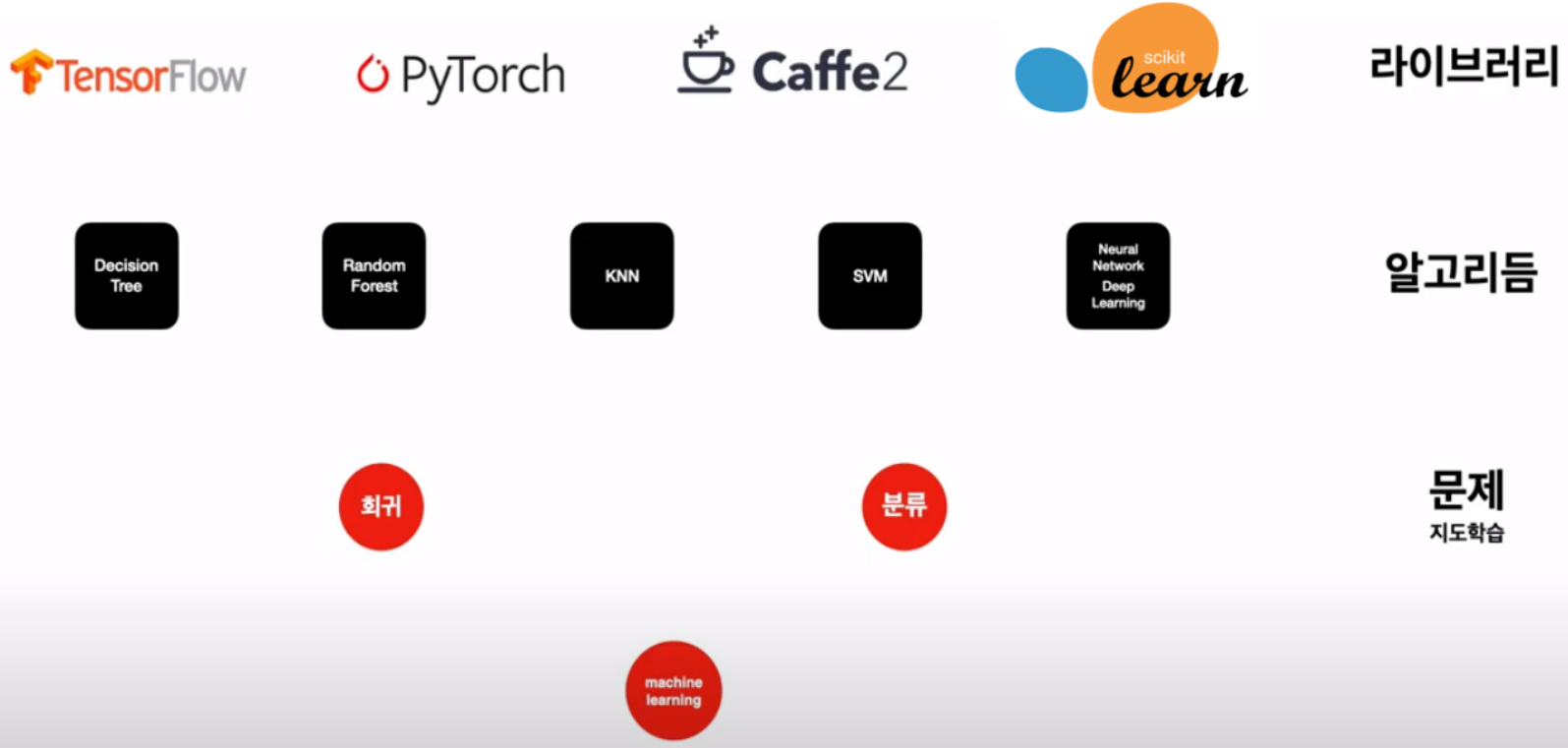
- 컴퓨터가 어떤 행동을 취할 때마다 외부에서 처벌이나 보상이 주어짐
- 알파고 최종 버전도 강화 학습 사용
- 게임에서 많이 사용됨



기계학습(ML) 알고리즘 선택



- Scikit-learn : 기계학습을 위한 라이브러리
 - 선형회귀, k-NN 알고리즘, 서포트 벡터머신, 랜덤 포레스트, 그래디언트 부스팅, k-means 등의 분류, 회귀, 클러스터링 알고리즘을 포함
 - 기계학습을 처음 접하는 사람들에게 좋은 도구



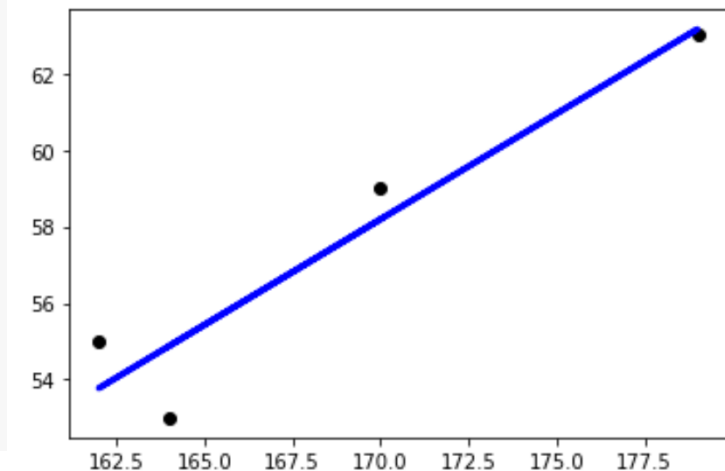
- 새로운 학생의 키를 전달하여, 몸무게 예측

```
1 import numpy as np
2 from sklearn import linear_model # scikit-learn 모듈을 가져온다
3
4 regr = linear_model.LinearRegression()
5
6 X = [[164], [179], [162], [170]] # 다중회귀에도 사용하도록 함
7 y = [53, 63, 55, 59]           # y = f(X)의 결과
8 regr.fit(X, y)
9
10 input_data = [ [180], [185] ]
11
12 result = regr.predict(input_data)
13 print(result)
```

☞ [63.71244635 66.47353362]

• 선형회귀 그래프 그리기

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3 from sklearn import linear_model # scikit-learn 모듈을 가져온다
4
5 regr = linear_model.LinearRegression()
6
7 X = [[164], [179], [162], [170]] # 선형회귀의 입력은 2차원으로 만들어야 함
8 y = [53, 63, 55, 59] # y = f(X)의 결과값
9 regr.fit(X, y)
10
11 # 학습 데이터와 y 값을 산포도로 그린다.
12 plt.scatter(X, y, color='black')
13
14 # 학습 데이터를 입력으로 하여 예측값을 계산한다.
15 y_pred = regr.predict(X)
16
17 # 학습 데이터와 예측값으로 선그래프로 그린다.
18 # 계산된 기울기와 y 절편을 가지는 직선이 그려진다
19 plt.plot(X, y_pred, color='blue', linewidth=3)
20 plt.show()
```



- 8명의 남녀학생에 대한 키와 몸무게로 선형 회귀 분석

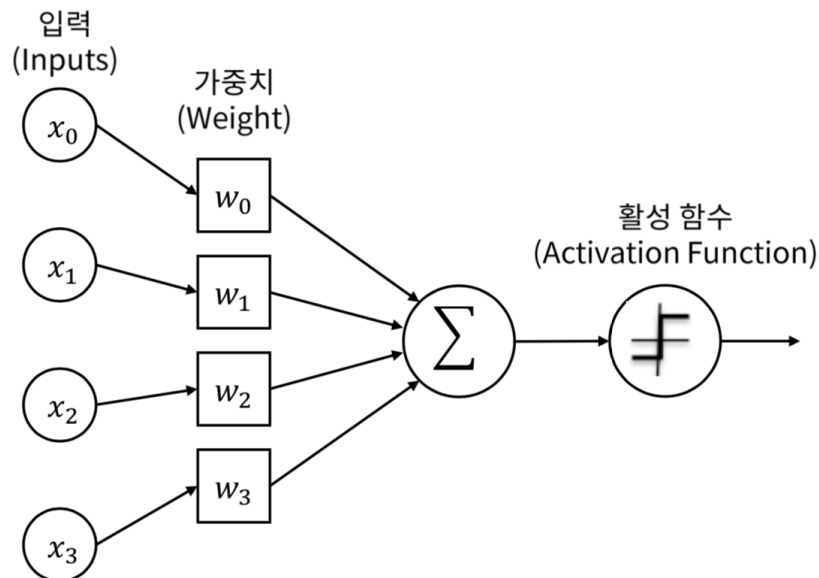
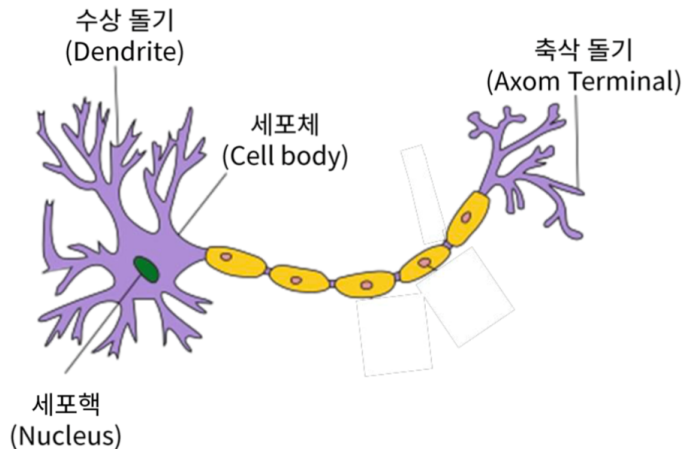
```
# 남자는 0, 여자는 1
# 입력데이터 X값은 2차원 배열로 정의
X = [[164, 1], [167, 1], [165, 0], [170, 0], [179, 0], [163, 1], [159, 0], [166, 1]]
# y 값은 1차원 데이터
y = [43, 48, 47, 66, 67, 50, 52, 44]
```

- 선형 회귀 분석 결과 출력
 - 계수
 - y 절편
- 키가 166인 여학생, 166인 남학생의 몸무게 예측 결과 출력

```
1 import numpy as np
2 from sklearn import linear_model
3
4 regr = linear_model.LinearRegression()
5
6 # 남자는 0, 여자는 1
7 # 입력데이터 X값은 2차원 배열로 정의
8 X = [[164, 1], [167, 1], [165, 0], [170, 0], [179, 0], [163, 1], [159, 0], [166, 1]]
9 # y 값은 1차원 데이터
10 y = [43, 48, 47, 66, 67, 50, 52, 44]
11
12 regr.fit(X, y)          # 학습
13 print('계수 :', regr.coef_)
14 print('절편 :', regr.intercept_)
15 print('점수 :', regr.score(X, y))
16 print('키 166 여학생의 추정 몸무게 :', regr.predict([[166, 1]]))
17 print('키 166 남학생의 추정 몸무게 :', regr.predict([[166, 0]]))
```

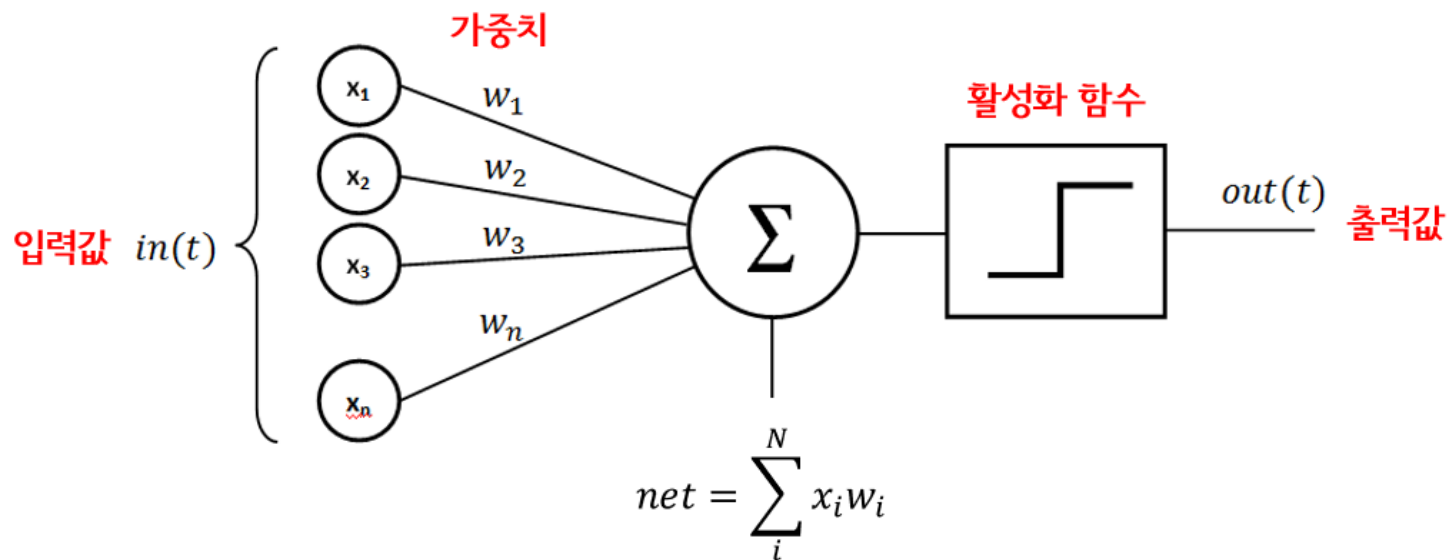
☞ 계수 : [0.88542825 -8.87235818]
절편 : -90.97330367074522
점수 : 0.7404546306026769
키 166 여학생의 추정 몸무게 : [47.13542825]
키 166 남학생의 추정 몸무게 : [56.00778643]

- 인간의 뇌를 흉내 내어 만든 기계 학습 모델
- 신경세포속 수상돌기는 다른 신경세포로부터 신호를 전달받아서(입력) 이 신호에 대한 반응을 축삭말단을 통해서 다른 신경세포로 전달(출력)
- 인공적인 신경세포를 흉내내는 프로그램을 퍼셉트론 (Perceptron) 이라고 함



퍼셉트론(perceptron)

- 입력 데이터는 입력 노드에 전달
- 입력 노드는 출력 노드로 연결
- 출력 노드는 두 개의 기능으로 구분
 - 전달되어 오는 신호를 모두 합하는 부분
 - 이렇게 합산된 신호에 따라 다음 계층으로 신호를 보낼 것인지 말 것인지는 활성화 activation 함수에 의해 결정



출처 : <https://liveyourit.tistory.com/63>

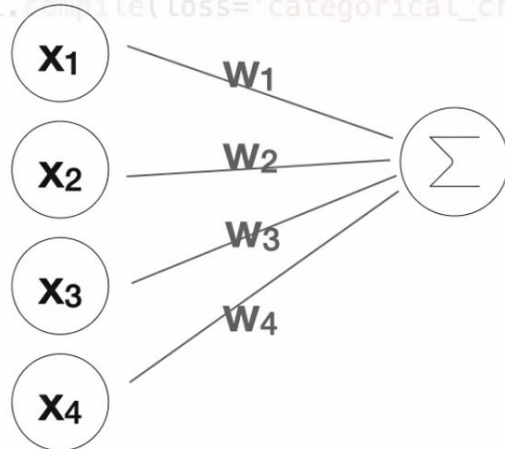
- 활성화 함수 종류

- 시그모이드(Sigmoid), 하이퍼볼릭 탄젠트(tanh), 소프트맥스(Softmax)
- 렐루(ReLU), 스위시(swish)

2. 모델의 구조를 만듭니다

```
X = tf.keras.layers.Input(shape=[4])  
Y = tf.keras.layers.Dense(3, activation='softmax')(X)  
model = tf.keras.models.Model(X, Y)  
model.compile(loss='categorical_crossentropy')
```

Activation 활성화 함수



$$y = f(w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b)$$

f

- 회귀모델: Identity ($y=x$)
- 분류모델: Softmax

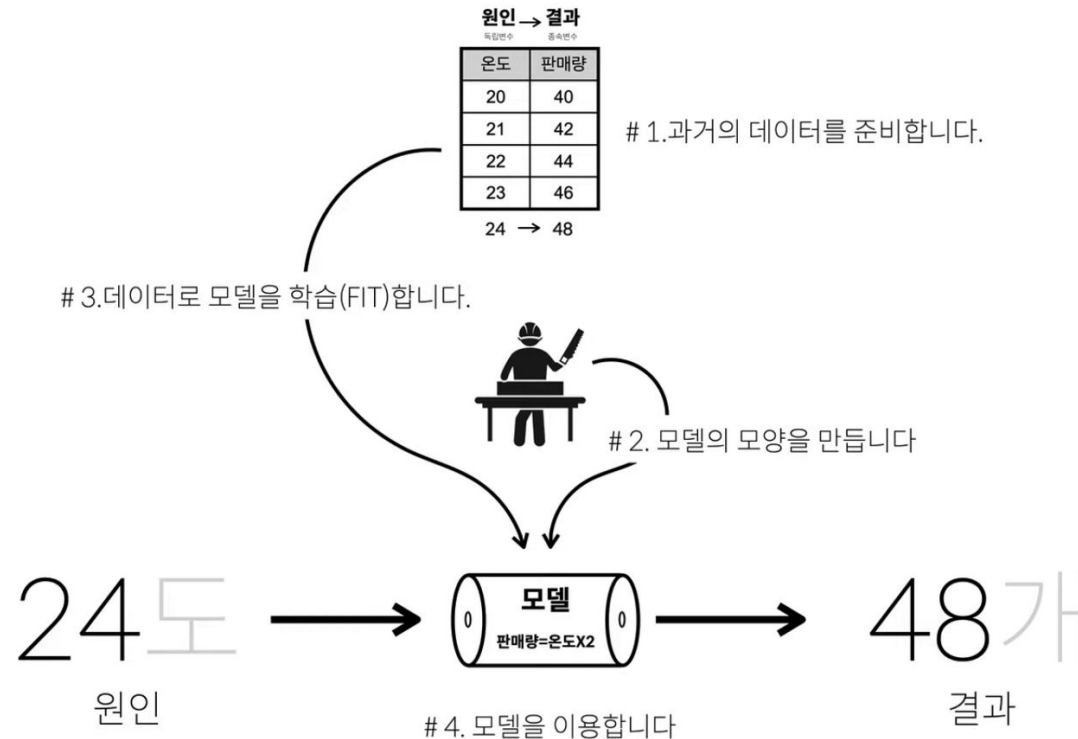
텐서플로우(Tensorflow)



- 머신러닝과 딥러닝을 위한 오픈소스 플랫폼
- 텐서플로우는 우수한 기능과 서비스를 제공
- 병렬처리를 잘 지원
- 고급 신경망 네트워크 모델을 쉽게 구현
 - 케라스 활용
- 텐서플로우 패키지 포함
 - `import tensorflow as tf`
- 텐서플로우 패키지 버전 확인
 - `tf.__version__`

머신러닝 활용 절차

- 데이터 준비
- 머신러닝 모델 구조 작성
- 기계 학습(Fit)
- 모델 이용



출처 : <https://opentutorials.org/module/4966/28967>

- 데이터 준비 → 판다스 이용
- 모델 구조 작성 → 케라스 이용

1. 과거의 데이터를 준비합니다.

```
레모네이드 = pd.read_csv('lemonade.csv')
```

```
독립 = 레모네이드[['온도']]
```

```
종속 = 레모네이드[['판매량']]
```

```
print(독립.shape, 종속.shape)
```

2. 모델의 구조를 만듭니다

```
X = tf.keras.layers.Input(shape=[1])
```

```
Y = tf.keras.layers.Dense(1)(X)
```

```
model = tf.keras.models.Model(X, Y)
```

```
model.compile(loss='mse')
```

3. 데이터로 모델을 학습(FIT)합니다.

```
model.fit(독립, 종속, epochs=1000)
```

4. 모델을 이용합니다

```
print("Predictions: ", model.predict([[15]]))
```

출처 : <https://opentutorials.org/module/4966/28967>

실습. 판매 예측(인공신경망, keras 활용)



```
1 import tensorflow as tf
2 import pandas as pd
3
4 # 1. 데이터를 준비합니다.
5 파일경로 = 'https://raw.githubusercontent.com/blackdew/tensorflow1/master/csv/lemonade.csv'
6 레모네이드 = pd.read_csv(파일경로)
7
8 # 종속변수, 독립변수
9 독립 = 레모네이드[['온도']]
10 종속 = 레모네이드[['판매량']]
11 print(독립.shape, 종속.shape)
12
13 # 2. 모델을 만듭니다.
14 X = tf.keras.layers.Input(shape=[1])
15 Y = tf.keras.layers.Dense(1)(X)
16 model = tf.keras.models.Model(X, Y)
17 model.compile(loss='mse')
18
19 # 3. 모델을 학습시킵니다.
20 model.fit(독립, 종속, epochs=1000, verbose=0)
21 model.fit(독립, 종속, epochs=10)
22
23 # 4. 모델을 이용합니다.
24 print(model.predict(독립))
25 print(model.predict([[15]]))
```



```
(6, 1) (6, 1)
Epoch 1/10
1/1 [=====] - 0s 5ms/step - loss: 0.0032
Epoch 2/10
1/1 [=====] - 0s 6ms/step - loss: 0.0032
Epoch 3/10
1/1 [=====] - 0s 6ms/step - loss: 0.0032
Epoch 4/10
1/1 [=====] - 0s 7ms/step - loss: 0.0032
Epoch 5/10
1/1 [=====] - 0s 9ms/step - loss: 0.0032
Epoch 6/10
1/1 [=====] - 0s 7ms/step - loss: 0.0032
Epoch 7/10
1/1 [=====] - 0s 8ms/step - loss: 0.0032
Epoch 8/10
1/1 [=====] - 0s 6ms/step - loss: 0.0032
Epoch 9/10
1/1 [=====] - 0s 8ms/step - loss: 0.0032
Epoch 10/10
1/1 [=====] - 0s 5ms/step - loss: 0.0032
[[40.094646]
 [42.06308 ]
 [44.031517]
 [45.999954]
 [47.96839 ]
 [49.93683 ]]
[[30.25246]]
```

지도 학습 : 회귀 vs 분류

- 지도 학습 데이터가 수치 데이터(양적)이면 회귀 문제
- 지도 학습 데이터가 범주형 데이터이면 분류 문제

온도	판매량
20	40
21	42
22	44
23	46

양적



회귀
regression

공부시간	시험결과
20	불합격
21	불합격
22	합격
23	합격

범주형



분류
classification

출처 : <https://opentutorials.org/module/4966/28967>

- 아이리스 품종은 범주형 데이터 → 분류 문제

- 원핫 인코딩 필요
 - 판다스의 get_dummies() 함수 이용

1. 과거의 데이터를 준비합니다.

```
아이리스 = pd.read_csv('iris.csv')
```

```
아이리스 = pd.get_dummies(아이리스)
```

```
독립 = 아이리스[['꽃잎길이', '꽃잎폭',  
                 '꽃받침길이', '꽃받침폭']]
```

```
종속 = 아이리스[['품종']]
```

```
print(독립.shape, 종속.shape)
```

2. 모델의 구조를 만듭니다

```
X = tf.keras.layers.Input(shape=[4])
```

```
Y = tf.keras.layers.Dense(3, activation='softmax')(X)
```

```
model = tf.keras.models.Model(X, Y)
```

```
model.compile(loss='categorical_crossentropy')
```

3. 데이터로 모델을 학습(FIT)합니다.

```
model.fit(독립, 종속, epochs=1000)
```

4. 모델을 이용합니다

```
print("Predictions: ", model.predict(독립[0:5]))
```

출처 : <https://opentutorials.org/module/4966/28967>

원핫 인코딩(onehot-encoding)

- 범주형 데이터를 수치 데이터로 변환

꽃잎길이	꽃잎폭	꽃받침길이	꽃받침폭	품종
5.1	3.5	1.4	0.2	setosa
4.9	3	1.4	0.2	versicolor
4.7	3.2	1.3	0.2	virginica
4.6	3.1	1.5	0.2	setosa

학습할 데이터를 준비합니다.

```
아이리스 = pd.read_csv('iris.csv')
```

원핫인코딩

```
아이리스 = pd.get_dummies(아이리스)
```

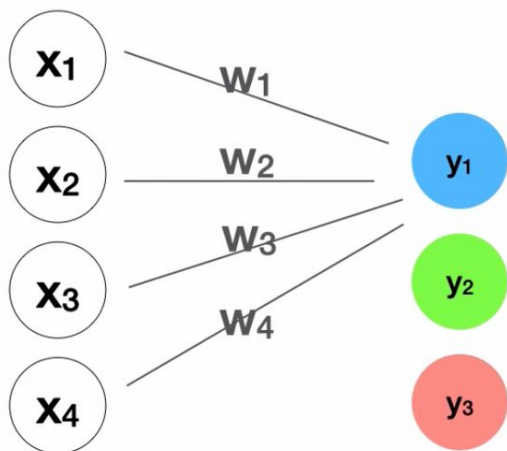
꽃잎길이	꽃잎폭	꽃받침길이	꽃받침폭	품종.setosa	품종.virginica	품종.versicolor
5.1	3.5	1.4	0.2	1	0	0
4.9	3	1.4	0.2	0	1	0
4.7	3.2	1.3	0.2	0	0	1
4.6	3.1	1.5	0.2	1	0	0

출처 : <https://opentutorials.org/module/4966/28967>

활성화 함수와 손실 함수

- 활성화 함수로 softmax 사용
- 손실(비용) 함수로 다중분류용 categorical_crossentropy 사용

꽃잎길이	꽃잎폭	꽃받침길이	꽃받침폭	품종.setosa	품종.virginica	품종.versicolor
5.1	3.5	1.4	0.2	1	0	0
4.9	3	1.4	0.2	0	1	0
4.7	3.2	1.3	0.2	0	0	1
4.6	3.1	1.5	0.2	1	0	0



2. 모델의 구조를 만듭니다

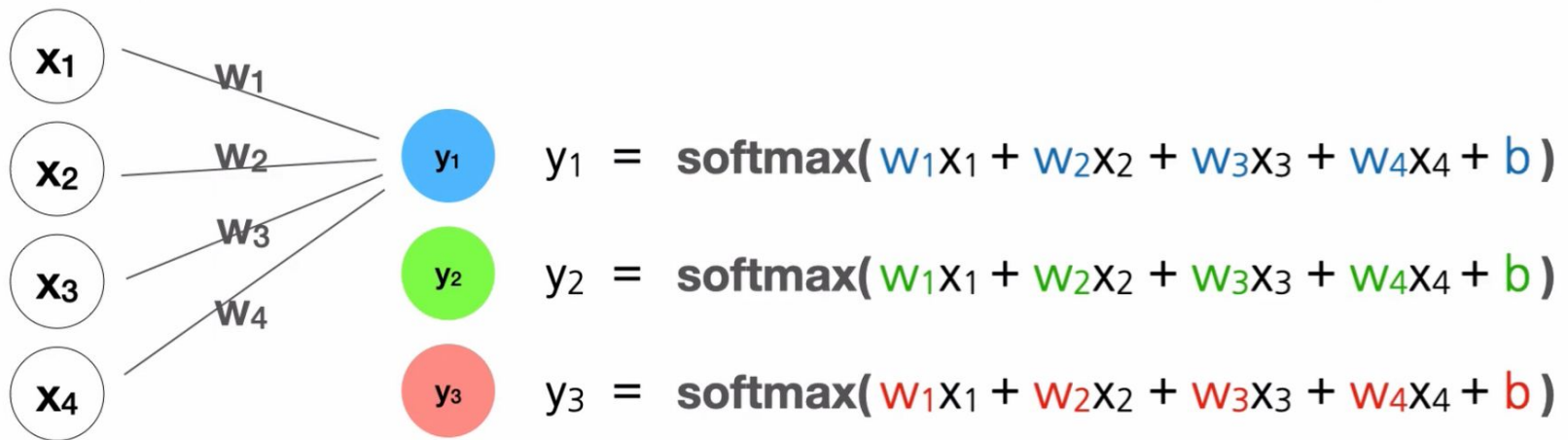
```
X = tf.keras.layers.Input(shape=[4])  
Y = tf.keras.layers.Dense(3, activation='softmax')(X)  
model = tf.keras.models.Model(X, Y)  
model.compile(loss='categorical_crossentropy')
```

출처 : <https://opentutorials.org/module/4966/28967>

소프트맥스(softmax)

- 다중 분류 문제에서 사용되는 활성화 함수
 - 확률 값의 합이 1

꽃잎길이	꽃잎폭	꽃받침길이	꽃받침폭	품종.setosa	품종.virginica	품종.versicolor
5.1	3.5	1.4	0.2	1	0	0
4.9	3	1.4	0.2	0.7	0.3	0
4.7	3.2	1.3	0.2	0.2	0.3	0.5
4.6	3.1	1.5	0.2	1	0	0



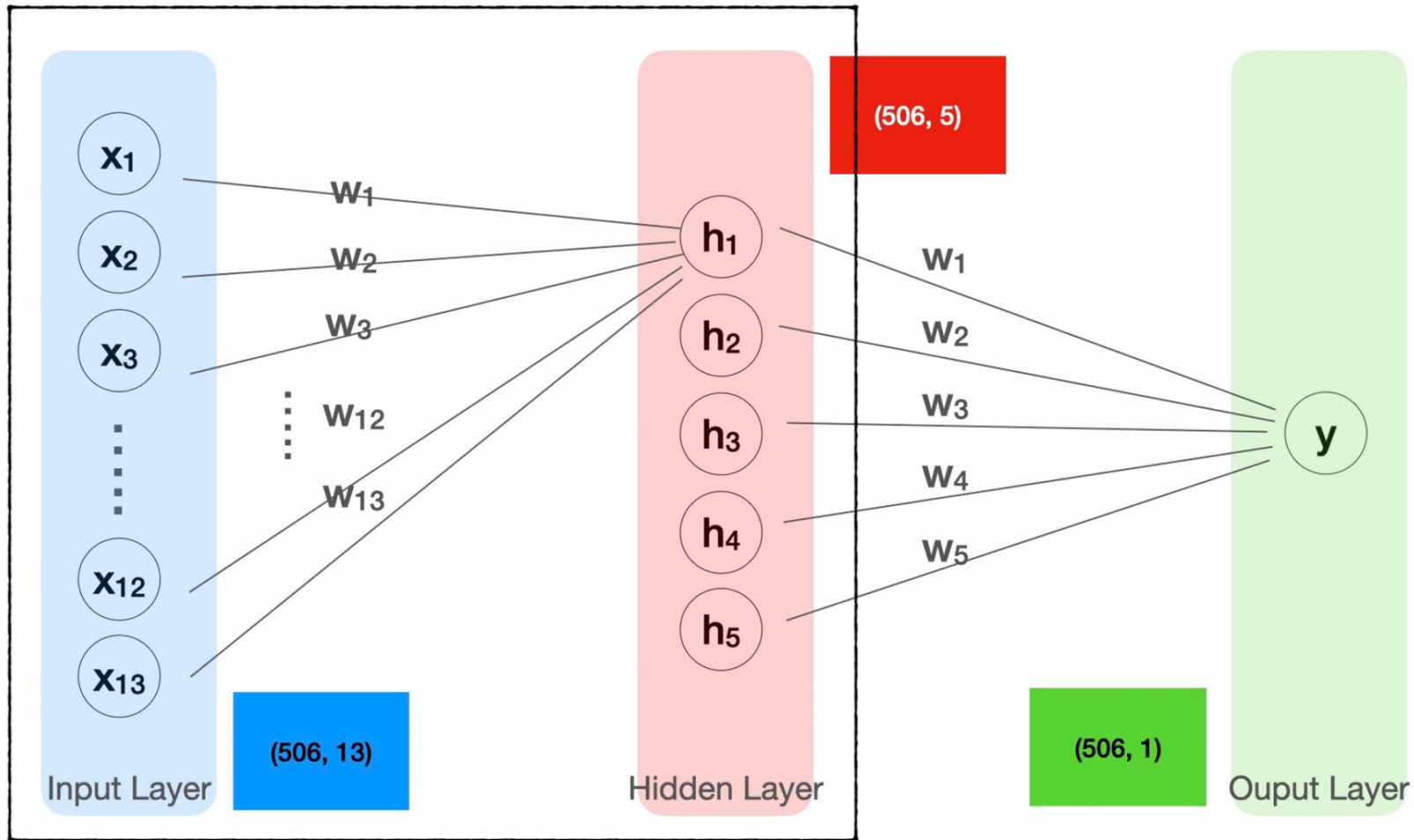
출처 : <https://opentutorials.org/module/4966/28967>

실습. 아이리스 품종 분류(인공신경망)

```
8 # 원핫인코딩
9 인코딩 = pd.get_dummies(아이리스)
10
11 # 독립변수, 종속변수
12 독립 = 인코딩[['꽃잎길이', '꽃잎폭', '꽃받침길이', '꽃받침폭']]
13 종속 = 인코딩[['품종_setosa', '품종_versicolor', '품종_virginica']]
14 print(독립.shape, 종속.shape)
15
16 # 2. 모델의 구조를 만듭니다
17 X = tf.keras.layers.Input(shape=[4])
18 Y = tf.keras.layers.Dense(3, activation='softmax')(X)
19 model = tf.keras.models.Model(X, Y)
20 model.compile(loss='categorical_crossentropy',
21               metrics='accuracy')
22
23 # 3. 데이터로 모델을 학습(FIT)합니다.
24 model.fit(독립, 종속, epochs=100)
25
26 # 모델을 이용합니다.
27 # 맨 처음 데이터 5개
28 print(model.predict(독립[:5]))
29 print(종속[:5])
```

```
5/5 [=====] - 0s 4ms/step - loss: 0.8970 - accuracy: 0.5333
Epoch 99/100
5/5 [=====] - 0s 5ms/step - loss: 0.8898 - accuracy: 0.5333
Epoch 100/100
5/5 [=====] - 0s 3ms/step - loss: 0.8844 - accuracy: 0.5400
[[0.480281  0.17675665 0.34296238]
 [0.54422194 0.17235859 0.2834195 ]
 [0.48229888 0.18219674 0.33550435]
 [0.46731317 0.19762081 0.33506605]
 [0.44463074 0.1835402  0.37182903]]
   품종_setosa  품종_versicolor  품종_virginica
0             1                0                0
1             1                0                0
2             1                0                0
3             1                0                0
4             1                0                0
```

- 히든 레이어를 여러 층 사용한 인공지능망



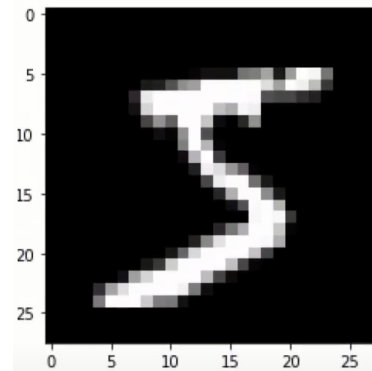
출처 : <https://opentutorials.org/module/4966/28967>

실습. 딥러닝으로 손글씨 숫자 분류

- MNIST 데이터 셋 활용
- 28*28 픽셀 이미지
- 총 70,000개 데이터로 구성
 - 훈련 데이터 60,000개
 - 테스트 데이터 10,000개



출처 : 위키백과(MNIST)



	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
5	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
6	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
7	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
8	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
9	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
10	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
11	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
12	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
13	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
14	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
16	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
17	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
18	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
19	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
20	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
21	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
22	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
23	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
24	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
25	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
26	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
27	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

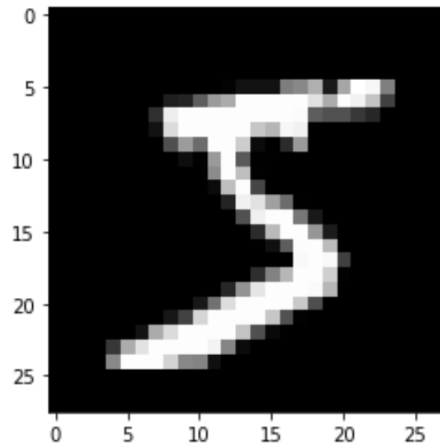
- MNIST 데이터 셋 불러오기

```
# 라이브러리 불러오기
import tensorflow as tf
from tensorflow import keras
import numpy as np
import matplotlib.pyplot as plt
```

```
#데이터 불러오기
mnist = keras.datasets.mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()
# x_train.shape = (60000,28,28), y_train.shape = (60000,1)
```

- 데이터 탐색

```
#훈련 데이터의 첫 번째 이미지 출력하기
plt.imshow(x_train[0], cmap = 'gray')
plt.show()
```



- 픽셀별 0~255 범위의 값을 0.0 ~ 1.0 범위의 값으로 변환(정규화)

0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.6	0.8	0.7	0.6	0.2	0.1	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.2	0.7	1.0	0.7	0.9	0.9	0.1	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.1	0.3	0.4	0.6	1.0	1.0	0.3	0.1	0.9	0.8	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.5	0.9	1.0	1.0	1.0	1.0	1.0	1.0	0.3	0.8	1.0	0.1	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.6	1.0	0.8	0.4	0.1	0.1	0.5	0.9	1.0	0.4	0.9	0.6	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.2	1.0	0.7	0.0	0.0	0.0	0.0	0.0	0.1	0.2	0.7	0.8	0.1	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.2	1.0	0.8	0.1	0.0	0.0	0.0	0.0	0.0	0.6	1.0	0.4	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.5	1.0	0.9	0.4	0.0	0.0	0.0	0.6	0.9	0.3	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.2	0.8	1.0	0.7	0.1	0.5	0.9	0.1	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.4	0.9	1.0	1.0	0.3	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.7	1.0	1.0	0.4	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.6	1.0	0.4	0.7	1.0	0.4	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.4	1.0	0.5	0.0	0.1	0.8	0.9	0.3	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.1	0.9	1.0	0.1	0.0	0.0	0.2	1.										

- 모델 구조 작성

- 입력층, 은닉층, 출력층 생성
- 입력층을 1차원으로 생성 : 2차원 이미지 배열을 1차원 이미지 배열로 변환
 - Flatten() 함수 사용
- 활성화 함수 설정
 - 은닉층 : 렐루(ReLU) 함수
 - 출력층 : 소프트맥스(softmax) 함수

```
model = keras.models.Sequential([
    keras.layers.Flatten(input_shape = [28, 28]),
    keras.layers.Dense(200, activation = 'relu'),
    keras.layers.Dense(100, activation = 'relu'),
    keras.layers.Dense(50, activation = 'relu'),
    keras.layers.Dense(10, activation = 'softmax')])
```

- 모델 컴파일

- 학습 방법 설정

- 손실 함수, 최적화 함수, 평가 지표 종류 선택

- 손실 함수

- 딥러닝의 **예측값과 실제값 사이의 차이(오차)**를 수치로 나타내는 함수
 - 종류
 - 이진 분류 모델 : binary_crossentropy
 - 다중 분류 모델
 - categorical_crossentropy : 정답 레이블이 **원-핫 인코딩 형태**일 때 사용
 - sparse_categorical_crossentropy : 정답 레이블이 **정수 형태**일 때 사용
 - 회귀 모델 : 평균제곱오차(MSE : Mean Squared Error)

- 최적화 함수

- 손실 함수의 값을 최소화하도록 **가중치와 바이어스 수정**해 나가는 함수
 - 종류
 - 경사하강법(Gradient Descent)
 - 확률적 경사하강법(Stochastic Gradient Descent)
 - Adam

- 모델 컴파일

- 평가 지표

- 모델이 결과를 얼마나 {잘 분류, 잘 예측} 하는지를 평가하는 지표
 - 종류
 - 분류 모델 : 정확도(Accuracy), 재현율(Recall), 정밀도(Precision)
 - 회귀 모델 : 평균절대오차(MAE : Mean Absolute Error)

- compile() 메소드 사용

- 최적화 함수 : adam
 - 손실 함수 : sparse_categorical_crossentropy
 - 평가 지표 : accuracy

```
model.compile(optimizer = 'adam', loss = 'sparse_categorical_crossentropy',  
              metrics = ['accuracy'] )
```

- 모델 학습

- 모델 컴파일에서 설정한 학습 방법으로 모델 학습
- 훈련 데이터, 훈련 데이터 레이블, 반복 학습 횟수(epochs) 설정 필요
 - 훈련 데이터를 반복 학습하면서 정확도를 높임
 - 너무 많이 학습하면 훈련 데이터만 정확히 예측하는 과대적합 문제 발생

- fit() 메소드 사용

```
model.fit(x_train, y_train, epochs = 5)
```

```
Epoch 1/5
1875/1875 [=====] - 5s 2ms/step - loss: 0.3940 - accuracy: 0.8843
Epoch 2/5
1875/1875 [=====] - 5s 3ms/step - loss: 0.0956 - accuracy: 0.9701
Epoch 3/5
1875/1875 [=====] - 5s 2ms/step - loss: 0.0633 - accuracy: 0.9797
Epoch 4/5
1875/1875 [=====] - 5s 3ms/step - loss: 0.0484 - accuracy: 0.9846
Epoch 5/5
1875/1875 [=====] - 5s 3ms/step - loss: 0.0407 - accuracy: 0.9873
<tensorflow.python.keras.callbacks.History at 0x2667b6f49a0>
```

• 모델 평가

- 모델이 학습 완료 후 테스트 데이터와 테스트 데이터 레이블을 사용하여 새로운 데이터를 얼마나 잘 예측하는지 확인
- evaluate() 메소드 사용

```
model.evaluate(x_test, y_test)
```

```
313/313 [=====] - 1s 2ms/step - loss: 0.0813 - accuracy: 0.9764  
[0.08130813390016556, 0.9764000177383423]
```

• 손글씨 숫자 예측

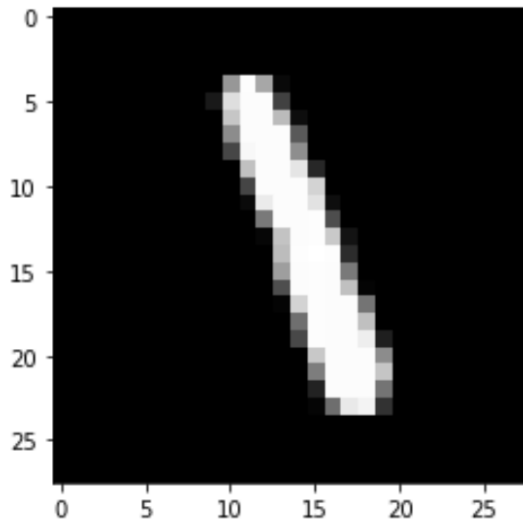
- predict() 메소드 사용

```
pred = model.predict(x_test)    # x_test.shape = (10000, 28, 28)  
print(type(pred), pred.shape)  
print(model.predict(x_test).round(2))
```

```
<class 'numpy.ndarray'> (10000, 10)  
[[0. 0. 0. ... 1. 0. 0.]  
 [0. 0. 1. ... 0. 0. 0.]  
 [0. 1. 0. ... 0. 0. 0.]  
 ...  
 [0. 0. 0. ... 0. 0. 0.]  
 [0. 0. 0. ... 0. 0. 0.]  
 [0. 0. 0. ... 0. 0. 0.]]
```

- 하나의 손글씨 숫자 데이터 예측

```
plt.imshow(x_test[46], cmap='gray')  
plt.show()
```



```
pred = model.predict(x_test[46].reshape(1,28,28)) # (28,28) => (1, 28, 28)  
print(type(pred), pred.shape)  
print(pred.round(2))  
print(np.argmax(pred), y_test[46]) # 정답과 비교
```

```
<class 'numpy.ndarray'> (1, 10)  
[[0. 1. 0. 0. 0. 0. 0. 0. 0. 0.]  
 1 1
```



thank you

본 과제(결과물)는 교육부와 한국연구재단의 재원으로 지원을 받아 수행된
디지털신기술인재양성 혁신공유대학사업의 연구결과입니다.